Python in Detail

How Does Misty's Python Library Work?

Misty's Python library is designed to directly use the API that Misty already hosts, acting as an API client for Misty. This API client is generated by interrogating Misty's API, and producing Python code as a result. The actual generator script can be found here on Github, and is the best place to start if you want to control Misty from another device. The generator has several dependencies, which can be seen in the setup.py script.

> **Note:** The method used to generate the API client is really straightforward and could be replicated in other programming languages without much difficulty. Misty exposes her API via a GetHelp command, which returns a complete list of supported functions and parameters.

After running the generator, the resulting updated mistyPy implementation will contain the full set of functionality appropriate for the version of robot used to generate that library. Internally, Misty uses this same method to generate her onboard Python API, which means that implementations built using it are compatible by simply changing the constructor for the Robot object. When running a script directly on Misty, an empty constructor implicitly connects to localhost.

Running on Misty vs a Development Computer

As a developer, you'll need to decide how you want to run your skill- your options are to run it from an external computer, or directly on Misty. Each method has pros and cons, but your requirements or preferred workflow may force one path or another.

Consider running your skill directly on Misty if:

- You need to use Misty's Python editor (for example, if you're using a Chromebook)
- You don't need any additional dependencies
- You want the skill to run using only the robot's resources
- You want the skill to run without network connectivity

Consider running your skill from an external computer if:

- The skill is complex, or requires multiple files
- The skill requires additional python libraries or dependencies
- The skill needs the computing power offered by the workstation

In addition, it generally makes things easier to build and debug if you code and test an implementation on your workstation, then move it to the robot for execution. Tools like Visual Studio Code offer a great debugging experience that won't be available to Misty Studio. A simple skill can generally be developed and tested from a workstation environment, then moved to Misty for execution.

How Does Misty Run a Python Script?

Misty does have some Python support, which allows her to directly run Python scripts. To achieve this, a Python script is sent as text to Misty. This is done by calling Misty's PythonStart API, passing in a script file. The script is then written to a file, and Misty executes it through through a shell script. Because of the way the script is executed, there are a couple of limitations to be aware of:

- The script must be a single file
- Only one script can be run at a time.

When the script is executed, stdout is piped to a subsequent file, and those results are made available. Because of this, standard print statements can be used to trace the execution of a program, or to capture specific outputs.

The relevant Python management functions are:

- PythonStart
  - Starts the provided Python script
  - REST API Command: POST http://{ROBOT-IP-ADDRESS}/api/python/start
- PythonStop
  - Stops a currently running Python script
  - REST API Command: POST http://{ROBOT-IP-ADDRESS}/api/python/stop
- PythonIsRunning
  - Checks if a Python script is currently executing
  - REST API Command: GET http://{ROBOT-IP-ADDRESS}/api/python/running
- PythonGetOutput
  - Gets the output of a completed Pytonh script
  - REST API Command: GET http://{ROBOT-IP-ADDRESS}/api/python/output

How Does the Misty Studio Python Editor Work?

Misty Studio also hosts a lightweight Python editor, which is useful for cases where an individual cannot install and maintain a local Python environment on their computer. While this isn't as fully featured as a full IDE, it does provide enough functionality to make for a good developer experience. This editor is built on top of the well known Monaco Editor, though it only supports a subset of the features.

To facilitate the development experience, Misty is also hosting a Python Language Server (PLS). This PLS is running on Misty, and provides support for code completion and syntax discovery. The PLS starts up with Misty, and must build an index of Misty's API and the available libraries prior to being ready to service the editor. Because this index must be created when the robot boots, the PLS won't be available for the first two minutes after a complete boot. When using Misty's Python editor, interactions with the PLS are handled via a websocket connection on port 5150.

> **Note:** Currently, the PLS on Misty only supports **ONE** client connection. What this means is that the most recent browser connection will receive a connection, and all others will be terminated. If code completion isn't working for you, check to make sure you're not working in multiple tabs.